Feel free to review the code in the actions.jsx file, which contains the core logic of this functionality. The composable nature of LangChain runnables offers a nice way to swap implementation details on the fly while keeping the rest of the application intact.

6.4.4 Overview of LangChain.js modules

We've seen how to use some of the modules available in the LangChain.js library. Table 6.3 summarizes these, and the main modules provide helpers and functions that work together to allow us to develop complex generative AI chains.

Table 6.3 The main modules available from the LangChain.js library

Module	Description
`langchain/prompts`	Manages prompt templates and input variables
`langchain/agents`	Implements autonomous agents that can use tools and make decisions
`langchain/chains`	Combines multiple components into reusable pipelines
`@langchain/core`	Core package that exposes many common utilities and functions
`@langchain/community`	Community packages
`langchain/vectorstores`	Various vector database store integrations
`langchain/storage`	Provides abstractions for storing data in memory or in databases
`langchain/document_loaders`	Provides helpers for loading documents from various sources
`@langchain/openai, @langchain/anthropic, ...`	Provides LLM integration classes with various providers
`langchain/output_parsers`	Exposes classes that transform outputs from LLMs into a more suitable format

6.5 Going deeper with LangChain.js

Two advanced LangChain.js features, the LCEL and LangGraph, provide greater control and flexibility for designing complex chains and workflows. They're not essential, but I'll introduce them here in case you want to look into them. LCEL provides a declarative, streamlined approach to building and optimizing chains for linear workflows, while LangGraph offers a more advanced, graph-based framework for creating stateful, multistep agentic applications with conditional logic and loops.

6.5.1 LangChain Expression Language

The LCEL is a declarative approach to building chains from existing `Runnables`. This means you describe the desired workflow, and LangChain optimizes the execution